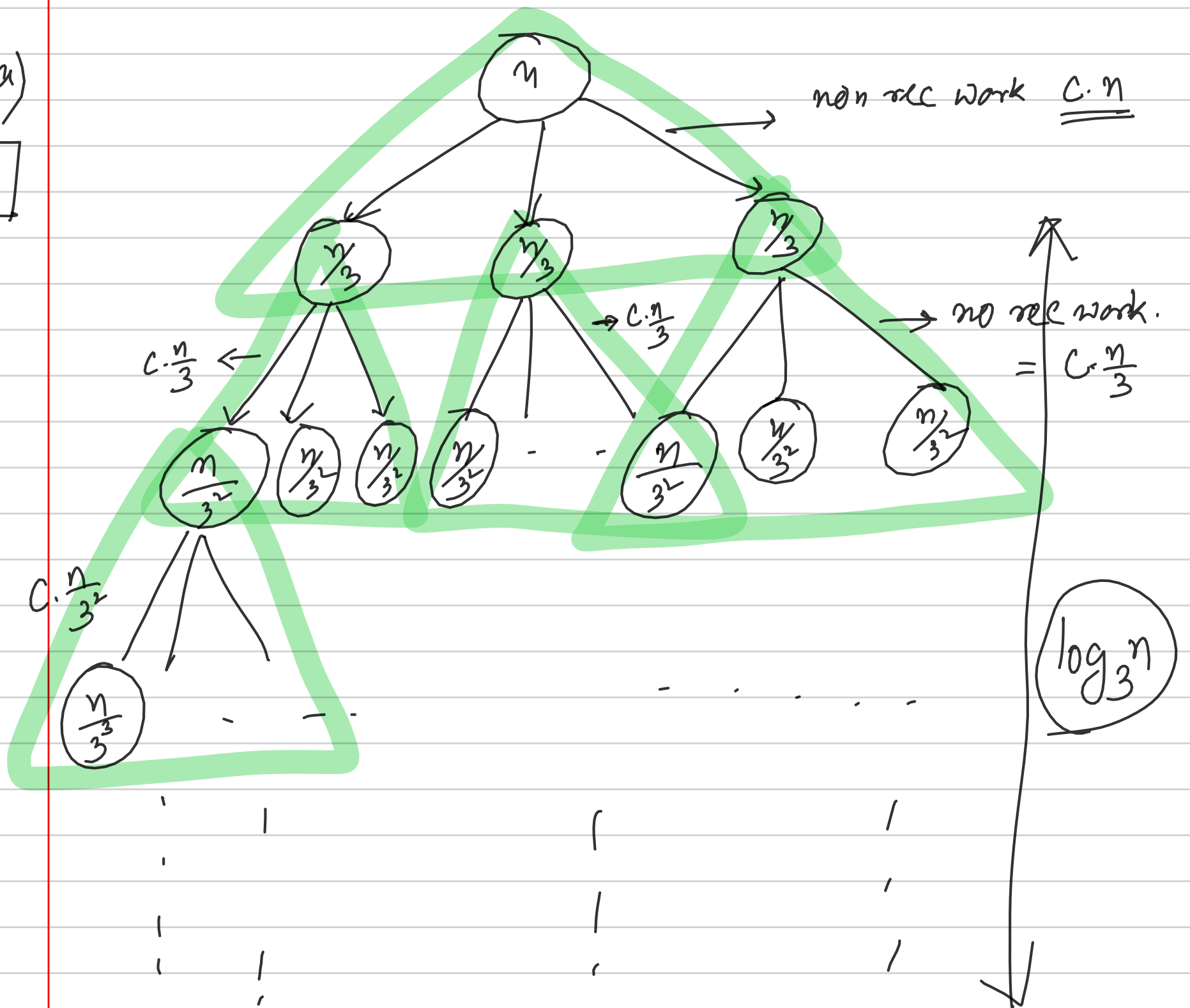


a)

$C \cdot n$

$C \cdot \frac{n}{3}$

$C \cdot n$



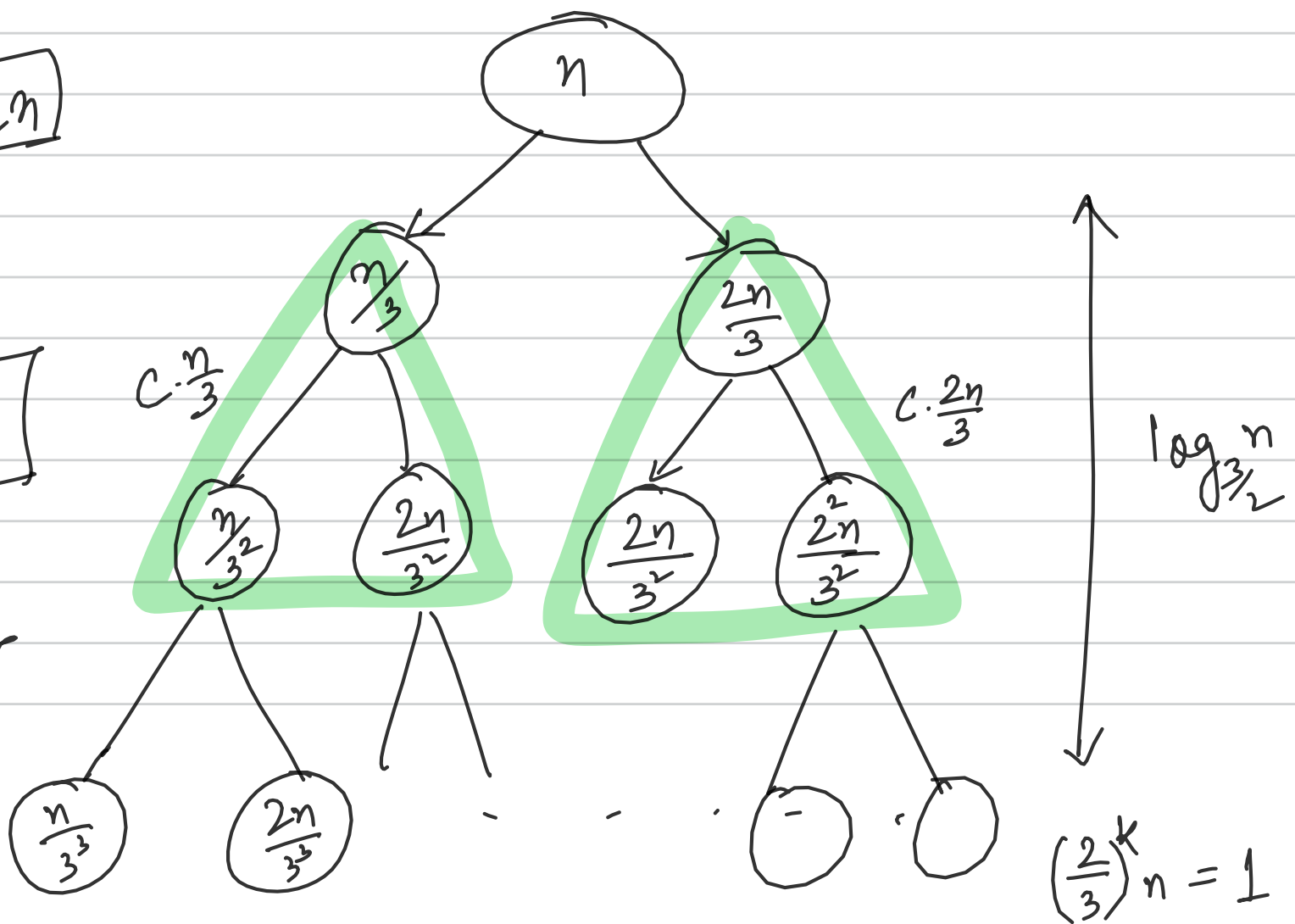
$$\text{Total} = C \cdot n \times \log_3 n = O(n \log n)$$

b)

Cn

$C \cdot n$

Cn



left most branch dies out first.
 right " " should be the longest

$\Rightarrow k = \log_{\frac{3}{2}} n$
 # level is
 decided by the
 right-most branch.

Total time = $O(n \log n)$

At Each step the array is divided into 1:99 ratio

$\Rightarrow$ Running time = $\underline{cn} \times \log_{\frac{100}{99}} n = O(n \log n)$

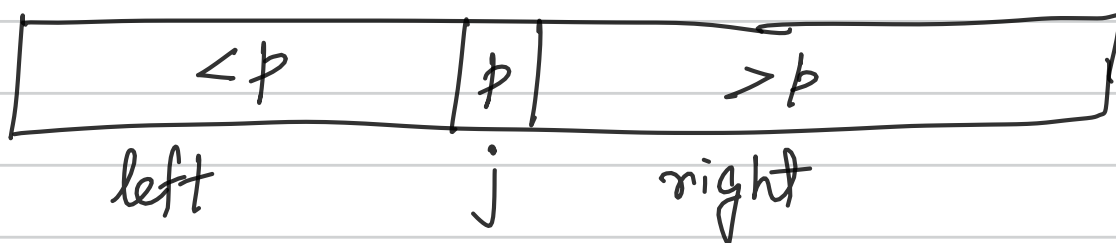
Quick Sort

Step 1. Choose one element and call that element "pivot".



first element as pivot.

Step 2: Partition the array A around pivot.

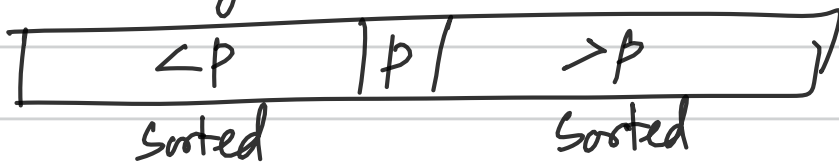


Explain later

Step 3. Sort the left subarray recursively.

Step 4

" " right " "

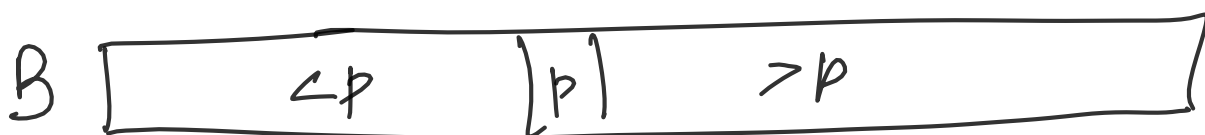


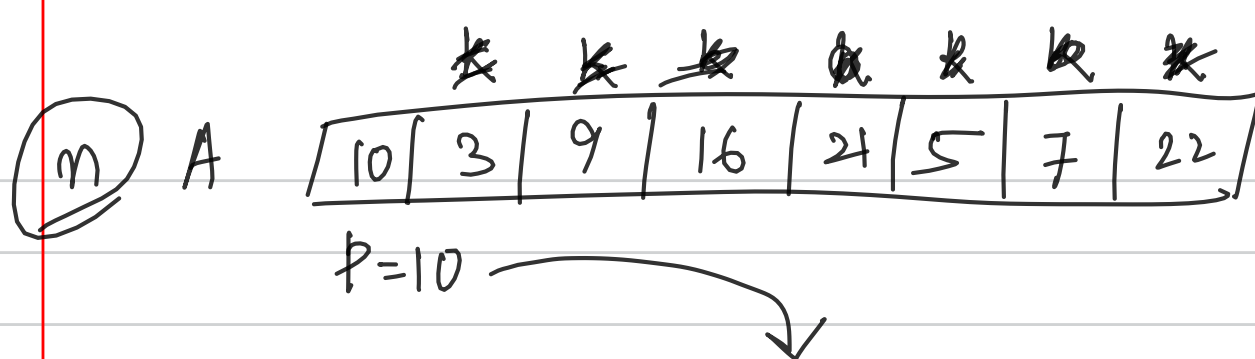
Step 5

We have our sorted array.

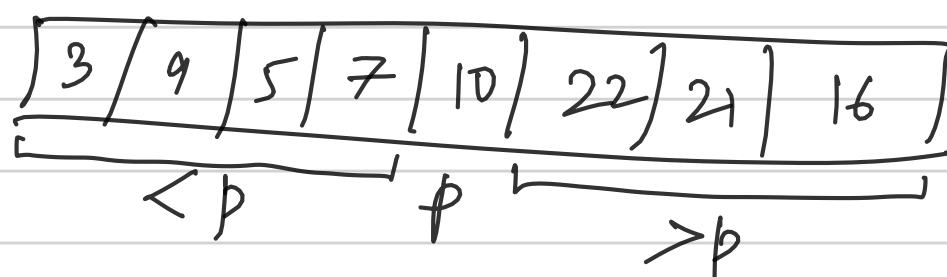
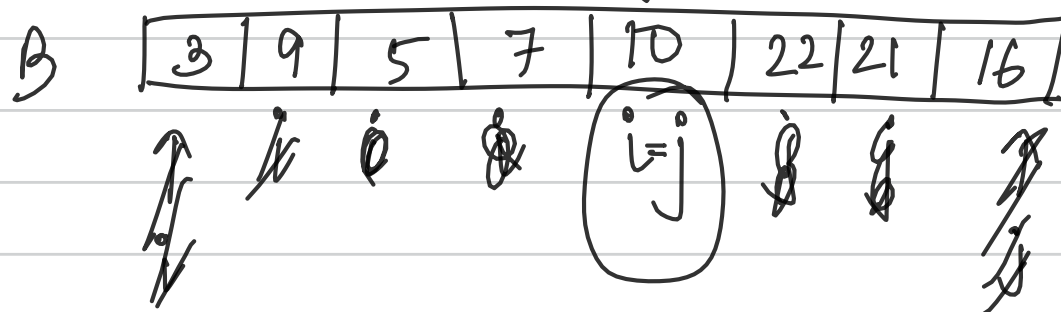
Step 2

Explain





$A[k] < p$



Running time for partition = $O(n)$

Return $\Rightarrow$ position of the pivot element.

Quick Sort (A, l, r)

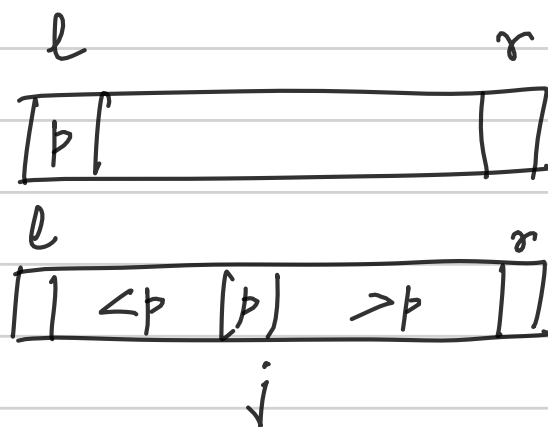
if ($l \geq r$), return

pivot = $A[l]$

$j = \text{partition}(A, l, r)$

Quick Sort ($A, l, j-1$)

Quick Sort ($A, j+1, r$)



Rec rel for the running time:

$$T(n) = T(k) + T(n-k-1) + O(n)$$

k ??
"
elements in left subarray.

Case I Suppose the pivot is always the median element

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \quad (\text{Same as merge sort})$$
$$O(n \log n)$$

Case II Suppose the pivot is such that it always makes a partition in $1:c$ ratio.

$$T(n) = T\left(\frac{n}{1+c}\right) + T\left(\frac{c}{1+c}n\right) + O(n)$$

$$T(n) = O(n \log n)$$

Case III Suppose the pivot is always the min element.

$$T(n) = T(n-1) + O(n)$$

$$T(n) =$$

$$T(n) \leq \underline{T(n-1)} + cn$$

$$\leq \underline{T(n-2)} + c(n-1) + cn$$

$$\leq T(n-3) + c(n-2) + c(n-1) + cn$$

$\vdots$

$$\leq \underline{T(1)} + c \left[2 + 3 + \dots + (n-1) + n \right]$$

$$= O(n^2)$$

Case IV The pivot always takes c element to the left and $(n-c)$ elements to the right.

$$T(n) = T(c) + T(n-c-1) + O(n)$$

$$T(n) = O(n^2)$$

(*) If constant fraction of the element is going in one part and rest in the other part, then

$$T(n) = O(n \log n)$$

(*) If constant number of elements are going in one part and rest in the other, then

$$T(n) = O(n^2)$$

Case II Again

1:99 ratio

How many pivot elements are there that gives

1:99 ratio or better ??

$$\frac{n}{100} : \frac{99n}{100}$$

no subarray should be of size $< \frac{n}{100}$



Smallest $\frac{n}{100}$

largest $\frac{n}{100}$

If the pivot belongs to smallest $\frac{n}{100}$ elements

then it's not 1:99 or better partition.

Similarly, If the pivot belongs to the largest $\frac{n}{100}$ elements

then also it's not 1:99 or better partition.

⇒ Any other element choose as pivot gives
1:99 or better partition.

1:9999

$$\frac{98n}{100}$$

$$\longrightarrow \underline{O(n \log n)}$$

$$\frac{9998n}{10000}$$

(*) There are many elements that gives $O(n \log n)$ running time if chosen as pivot.

There are ~~are~~ less elements that gives $O(n^2)$ running time if choose as pivot.

Randomized Quick Sort

(*) Step 1: Choose the pivot element uniformly at random

{ we can choose any element
chances of choosing any
element as pivot are the same.

→ Expected Running time = $O(n \log n)$

Arg